rvLLM: Single-GPU, FP8, Graph-Captured LLM Inference in Rust

Andy Norris (m0at)

`https://github.com/m0at/rvllm`

San Francisco, April 16, 2026

Abstract

rvLLM is a single-GPU LLM inference engine written in Rust, built from scratch without Python in the serving hot path. On a single NVIDIA H100 SXM 80 GB running Qwen2.5-7B in FP8 E4M3 with greedy decoding and a single CUDA-graph replay per step, rvLLM v3 reaches **22,496 tokens/second at** $N = 128$ (decode + LM head + argmax, 30 iterations, 5 warmup). For direct comparison, vLLM 0.19 (V1 engine, FP8, FlashInfer, CUDA graphs on) running on the same GPU with the same model and batch size reports **19,399 tok/s** as its steady-state generation throughput; rvLLM v3 is **1.16× faster** at the kernel level. The earlier v2 stack peaked at 19,287 tok/s at the same batch (SHA `eb9e247fd`) and is kept in-tree as an archive.

v3 ships as a FlashAttention-3 SM90 paged-KV decode kernel (built from the Hopper source to `libfa3_kernels.so`), five FP8 linears driven by `cublasLtMatmul` with fused bias and residual epilogues, a set of custom fused PTX kernels (add+RMSNorm+quantize, RoPE + KV-cache write, SiLU·mul+quantize, per-token quantize, argmax), and a graph-capture runtime that records **339 kernel launches per decode step into a single `cuGraphLaunch`**. Q, K, and V projections are fused into a single GEMM with bias epilogue (one call replaces three GEMMs and a bias kernel). No megakernels: each launch is a recognizable composite.

The paper states the engine's correctness invariants and enumerates the structural bug classes that rvLLM is designed to eliminate: schedule-epilogue pairing mismatches in SM90 CUTLASS FP8 GEMMs (silent `CUDA_ERROR_ILLEGAL_ADDRESS` under graph replay only), captured metadata offsets clobbered by orthogonal upload paths, stale paged-KV block tables from CoW-only change detection, and silent correctness loss from omitting attention biases present in the model (Qwen2.5 `attention_bias=true`). Each is eliminated by a pinned variant, a frozen layout, a typed signal from the scheduler, or an explicit tensor load. Missing build artifacts refuse to start; no silent fallbacks exist in the FP8 dispatch. rvLLM is open-source under Apache-2.0.

1 Introduction

Inference serving for large language models is bandwidth-bound at decode batch sizes of practical interest. On an H100 running Qwen2.5-7B, a single decode step involves roughly 10–15 kernel launches per transformer layer, paged-KV attention over the sequence's current context, and a final projection to a 152,064-entry vocabulary. At a batch size of 128 and a sequence length of order 100 tokens, throughput is dominated by GEMM kernel efficiency, HBM traffic, and per-step launch overhead.

vLLM [1] established the paged-KV cache design and continuous batching scheduler that define the modern serving baseline. CUTLASS 3.x [**?**] and FlashAttention-3 [**?**] supplied the kernel primitives that make Hopper-specific WGMMA and TMA accessible to non-Python callers. rvLLM composes these primitives behind a Rust runtime that owns CUDA graph capture, paged KV allocation, metadata packing, and sampling DtoH coordination in 31 crates (∼76K lines of Rust), with no Python interpreter in the serving path.

This paper documents rvLLM v3, which achieves 22,496 tok/s at $N = 128$ on a single H100 under a bench that re-uploads metadata each step and runs a 16-step faux-prefill (fair-comparison configuration, Section 7). The previous v2 stack (archived at commit `eb9e247fd`, April 16, 2026) reached 19,287 tok/s; vLLM 0.19 V1 on the same box measures 19,399 tok/s. v3 is 1.16× faster than both on the same GPU with the same model, same FP8 quantization, same CUDA-graph configuration. v3's primary levers are: (i) fusing Q/K/V into a single GEMM where v2 runs three; (ii) routing all five FP8 linears (QKV, O, gate_up, down, lm_head) through `cublasLtMatmul`, whose per-shape heuristic explores many more algorithms than a 40-variant CUTLASS cache; (iii) fusing the QKV row-bias into the GEMM epilogue via `CUBLASLT_EPILOGUE_BIAS`, and the residual add into the O and down GEMMs via `beta`=1; (iv) f16 RoPE tables, eliminating f32 from the decode hot path.

Section 2 walks the v3 stack. Section 3 enumerates the kernel inventory. Section 4 gives an ordered trace of one decode step. Section 5 states the correctness invariants and ties each to a concrete failure mode observed during development. Section 6 preserves the v2 recovery history (9,531 → 19,287 tok/s in a one-day root-cause pass) and then describes the v3 session (551 → 22,496 tok/s). Section 7 reports the full bench table.

2 The stack, every layer

```
HTTP / OpenAI API          rvllm-api (axum)
Engine                     rvllm-v2::engine
                           step_pipelined() = launch + collect
                           double-buffered pinned argmax DtoH
Scheduler                  continuous batching
                           paged KV (block_size = 64)
                           page-boundary growth signal
Worker                     CUDA graph pool
                           35 pre-captured batch buckets
                           1 compute stream per worker
Runner                     1 HBM arena, pre-allocated slab
                           packed metadata (1 HtoD per step)
Layer                      11 kernel launches per layer
Kernels                    CUTLASS 3.x SM90 FP8 GEMM
                           FlashAttention-3 SM90 paged decode
                           custom fused PTX (norm / silu / rope)
                           cuBLAS HGEMM (LM head)
```

Figure 1: The rvLLM stack. Each row runs in a single Rust crate; the DAG is enforced by a compile-time test that parses every `Cargo.toml` and rejects out-of-layer dependencies.

2.1 HTTP, engine, scheduler

HTTP is an `axum` server exposing OpenAI-compatible `/v1/completions`, `/v1/chat/completions`, `/v1/embeddings`, and `/v1/responses`. Incoming requests are tokenized and placed on a lock-free MPSC queue consumed by a single worker thread; the worker is `!Send` by construction so a tokio runtime cannot accidentally schedule two concurrent GPU users.

The engine (`rvllm-v2::engine`) exposes two functions: `step_launch(diff)` enqueues all GPU work for one step (metadata HtoD, graph replay, argmax DtoH, event record) and returns immediately; `step_collect()` waits on the previous step's DtoH event and returns the new tokens. There is exactly one codepath; the synchronous variant that tied `cuStreamSynchronize` to each step has been removed.

The scheduler (`rvllm-scheduler`) implements continuous batching and paged-KV allocation. A request moves through `Queued` → `Prefilling` → `Decoding` → `Finished`. The scheduler

emits one `BatchPlan` per step, of exactly one variant (`Prefill`, `Decode`, or `Idle`); prefill and decode never coexist in the same step. A watermark-triggered preemption path re-queues the lowest-priority decoding requests when KV-cache block use exceeds 96%.

2.2 Worker, runner, layer

The worker (`rvllm-worker` / `rvllm-v2::worker`) owns one CUDA compute stream, one CUDA context, and a pool of pre-captured graph instances indexed by bucket size. Buckets are $\{1, 2, 4, 8, 16, 24, 32, \ldots, 256\}$. Graph capture happens during engine initialization, not during warmup.

The runner (`rvllm-v2::runner`) owns one HBM arena allocated via one `cuMemAlloc` at startup, sized for the worst-case workspace (sweep over all CUTLASS variants $\times$ all bucket sizes $\times$ all GEMM shapes), plus the KV cache, scratch tensors, and packed metadata buffer. A compile-time `GraphSafe` trait marks tensors whose lifetime outlives the capture region; `&mut HbmArena` is not in scope inside a `CaptureScope` closure, so runtime realloc is unrepresentable.

The layer (`rvllm-v2::layer`) is a pure function of (input, weights, KV cache slab, scratch, metadata handle). One decode layer at FP8 is eleven kernel launches:

```
1.  fused_add_rmsnorm_fp8_quant
2.  CUTLASS FP8 GEMM (QKV)
3.  fused_rope_kv_write
4.  FA3 paged_decode
5.  quantize_fp8_per_token (post-attention)
6.  CUTLASS FP8 GEMM + residual (o_proj, v1)
7.  fused_rmsnorm_fp8_quant
8.  CUTLASS FP8 GEMM (gate_up)
9.  fused_silu_mul_fp8_quant
10. CUTLASS FP8 GEMM (down_proj)
11. residual_add_f16
```

Buffer aliasing is checked by the borrow checker: the layer takes `&mut Tensor<'a, f16>` for its output and `&mut LayerScratch` for scratch; two mutable borrows into one allocation fail to compile.

3 Kernel inventory

Every kernel has a pinned variant and a workspace contract. There are no dispatch fallback chains: each call site resolves to exactly one variant, determined at engine init from the autotune policy or pinned in source.

3.1 CUTLASS SM90 FP8 GEMM

CUTLASS 3.x templates drive the QKV, gate_up, down_proj, and fused o_proj+residual projections. Forty non-residual and ten residual-fused variants are instantiated, each parameterized by (TileShape, ClusterShape, KernelSchedule).

Variants cover tile shapes $\{64{\times}128, 128{\times}128, 128{\times}256\}$ in $M{\times}N$ with $K = 128$ or 256; cluster shapes $1{\times}1{\times}1$; and the `KernelTmaWarpSpecialized` (WS), `KernelTmaWarpSpecializedCooperative` (Coop), and their `FP8FastAccum` variants. Per-row activation scale and per-tensor weight scale are applied in the epilogue; the residual-fused kernel adds an `Sm90SrcFetch` fetch of the residual and a `TreeVisitor` computing $D = \alpha \cdot \mathrm{GEMM}(A, B) + C$.

Pairing constraint. The mainloop schedule must match the epilogue schedule. The residual-fused kernel uses `TmaWarpSpecializedCooperative` in its epilogue collective (`kernels/cutlass_fp8_gemm_res` line 100); any variant passing a non-cooperative mainloop with this epilogue causes `CUDA_ERROR_ILLEGAL_ADDRES` *only under graph replay* (the kernel runs cleanly outside a captured graph). The v2 runtime pins the o_proj residual path to variant `v1` ($128 \times 128 \times 128$ Coop/Coop matched); the v3 rewrite makes the mismatch a CUDA `static_assert`, rejecting the offending template pair at compile time.

Workspace contract. Each variant exposes `fp8_gemm_v<i>_workspace_size(M,N,K)`; the runtime's `max_workspace_size` query iterates all actually-callable variants across all bucket sizes for all GEMM shapes the model uses, and pre-allocates a single workspace slab accordingly. This eliminates the class of bug where the dispatch falls through to a variant whose workspace was never queried.

Autotune policy. A separate binary, `autotune-cutlass`, runs 67 HGEMM variants, 31 o-proj-residual variants, 32 gate-up-silu variants, 40 FP8 GEMM variants, and 10 FP8-GEMM-residual variants against every GEMM shape the model uses at every bucket size, reporting median of 10 timed runs after 3 warmups. The winning (shape $\rightarrow$ variant index) mapping is written to `policy.json`, which is a build artifact shipped alongside the binaries. The runtime does not consult `~/.cache` for policy; stale caches from prior deploys cannot influence dispatch.

3.2 FlashAttention-3 SM90 paged decode

`libfa3_kernels.so` is built at deploy time from the FlashAttention-3 Hopper source tree plus an `extern "C"` wrapper (`kernels/fa3_sm90_wrapper.cu`). Build time is roughly 10 minutes on an H100; the resulting shared object is 23 MB. The wrapper exposes `paged_decode` and `paged_prefill` entry points. Both expect KV laid out as [2, num_blocks, block_size, num_kv_heads, head_dim in one contiguous `Region`; the factor-of-two comes from interleaving K and V per block.

GQA is supported via num_heads/num_kv_heads; `head_dim = 128` is a hard gate (the kernel is specialized and refuses other head dimensions). `context_lens[i] == 0` is a valid padded-slot marker and yields zero output without touching `block_tables[i,*]`.

No PTX fallback. A prior PTX-only attention kernel existed in the tree. It has been removed from the dispatch; if `libfa3_kernels.so` is not present at engine init, the engine refuses to load. The build script ships with the repository.

3.3 Fused pre/post kernels

Eight custom CUDA kernels compile to PTX and are loaded by the runtime. Each fuses one recognizable composite. No megakernels exist in the dispatch path.

Kernel	Fuses	Launches
`embedding_gather`	token_ids $\rightarrow$ f16 hidden	$1 \rightarrow 1$
`fused_add_rmsnorm_fp8_quant`	residual + RMSNorm + quant	$3 \rightarrow 1$
`fused_rmsnorm_fp8_quant`	RMSNorm + quant	$2 \rightarrow 1$
`quantize_fp8_per_token`	post-attention act $\rightarrow$ FP8	$1 \rightarrow 1$
`fused_rope_kv_write`	RoPE + paged KV write	$3 \rightarrow 1$
`fused_silu_mul_fp8_quant`	SiLU + mul + quant	$3 \rightarrow 1$
`argmax`	f32 logits $\rightarrow$ i32 token	$1 \rightarrow 1$
`residual_add_f16`	$x + y$ in place	$1 \rightarrow 1$

Each fused kernel ships with a pure-Rust f32 reference implementation in tests; the PTX must match within cosine 0.999 for FP8 outputs and 10^{-5} absolute for f32 outputs, over a fuzz of

shape and value distributions. The `residual_add_f16` kernel is deliberately *not* fused into the GEMM epilogue; prior attempts to fuse residual with the downstream projection hit either schedule-pairing crashes or a ∼24% regression from increased tile pressure.

Vectorization rule. Kernels that accept dimensions divisible by eight halves issue `uint4` 128-bit loads (eight halves per load) and `uint2` 64-bit FP8 stores (eight FP8 per store), with register caching between the absolute-max reduction pass and the quantize pass so each HBM element is read once. Alignment is checked by a `debug_assert` in the Rust binding; misalignment fails the launch loudly rather than producing `CUDA_ERROR_MISALIGNED_ADDRESS` under graph replay.

4 One decode step, in order

For a decode batch of N sequences at bucket size $B = \text{pad_up}(N)$:

```
step_launch(diff):
  1. schedule(diff)                          CPU, O(N), zero allocs
  2. metadata pack + 1x HtoD                 packed i32 buffer:
                                             positions, context_lens,
                                             block_tables, slot_mapping
  3. graph_pool.replay(bucket = B)           one cuGraphLaunch:
       embedding_gather                       1 kernel
       for layer in 0..28:                    11 kernels x 28 = 308
         (as enumerated in Section 2.2)
       fused_residual_rmsnorm (final)         1 kernel
       cuBLAS HGEMM -> lm_head                1 kernel
       argmax                                 1 kernel
  4. cuMemcpyDtoHAsync argmax -> pinned[w]   4*N bytes (512 B at N=128)
     cuEventRecord event[w]
  5. return; w ^= 1                          double-buffer flip

step_collect():
  1. cuEventSynchronize event[r]             waits for step-1 DtoH
  2. read pinned[r][0..N]                    CPU reads new tokens
  3. scheduler.commit()                      mark tokens, free finished
```

Total kernel launches per decode step: $1 + 28 \times 11 + 1 + 1 + 1 = \mathbf{311}$, all captured into **one** `cuGraphLaunch`. Total DtoH per step: one $4 \cdot N$ byte copy of argmax token identifiers. Total HtoD per step: one packed metadata buffer, <100 KB at $N = 128$ with max_blocks $= 129$.

The LM-head GEMM uses cuBLAS HGEMM (F16 in, F32 out) because CUTLASS FP8 autotune reported worse time on the $(N, 152064, 3584)$ shape at $M \geq 128$ than cuBLAS for the model tested. An FP8 LM-head path exists in `rvllm-v2` gated on the autotune cache's preferred variant; it is disabled by default for Qwen2.5-7B.

5 Correctness invariants

The engine enforces five invariants at build or startup time. Each is a response to a concrete failure mode observed during development.

5.1 No fallbacks

Rule. Missing autotune-policy entry for a shape panics with the shape. Missing `libfa3_kernels.so` refuses startup. Missing `libcutlass_kernels.so` refuses startup. The runtime does not

read ~/.cache/rvllm/cutlass_autotune.json; policy lives in the SHA-pinned policy.json shipped alongside the binary.

Failure mode this prevents. A prior deploy populated the on-box autotune cache with entries for an older CUTLASS catalog. A fresh build with the same HF-downloaded kernels inadvertently consulted the stale cache, resolving shape $(32, 152064, 3584)$ to a variant whose template no longer matched the .so's symbol table, and ran the corresponding FP8FastAccum kernel under graph capture. Under graph replay, the kernel faulted with CUDA_ERROR_ILLEGAL_ADDRESS. SHA-pinning the policy artifact eliminates the failure class.

5.2 Single metadata-upload path

Rule. Metadata buffer layout is frozen per $(\text{bucket}, \text{max_blocks_per_seq})$. Captured graphs bind those exact device offsets. There is no non-padded upload variant. Prefill and decode use separate entry points that do not share offsets.

Failure mode. During development, two upload paths coexisted: upload_metadata (non-padded, sized to the actual request count) and upload_metadata_padded (sized to the captured bucket). On the first decode step after a prefill, the runtime's last_meta_offsets field carried the non-padded offsets from the prefill call, but the captured graph still read from the padded offsets. The diff-upload path wrote new token IDs, positions, and block-table entries at the non-padded indices; the captured kernel interpreted the bytes at the padded indices as if they were current data, producing a context_lens value of order 10^5, a total_tiles of 780, and a block_tables read 391 entries past the slice end. The read resolved to an uninitialized region of the meta buffer, which held bytes of a small integer interpreted as phys_block. Multiplied by the per-block stride 16384, this yielded a device address 10.2 MB before the nearest valid allocation—verbatim the compute-sanitizer report of the original crash.

The v3 rewrite removes non-padded upload entirely. The v2 engine tracks last_padded_batch: Option<usize> and only takes the patch fast-path when it matches the requested bucket; otherwise a full padded upload happens.

5.3 Real block-change detection

Rule. The scheduler emits ContinuedRequest::block_table_update: Option<Vec<BlockId>> whenever a sequence's physical block list has grown since the last mark_table_sent(). The worker unions this with diff.block_ops.copies (copy-on-write events) to compute has_block_changes before each patch-decode upload.

Failure mode. An earlier version of the worker checked only !diff.block_ops.copies.is_empty(), which captures CoW events but *not* ordinary page-boundary growth. A sequence reaching the end of its current block and being allocated a new physical block generates no CoW event; the block-table entry at index $\lceil \text{context_len}/\text{block_size} \rceil - 1$ changes silently. The patch-decode path skipped the block_tables copy; the captured graph continued to read the stale block identifier, producing wrong KV-cache reads, garbage attention outputs, and garbage tokens. No crash. ignore_eos=true bench runs at output_len 512 still reported plausible throughput; correctness was degraded silently. The scheduler already emitted the needed signal; the worker simply did not consult it.

5.4 CUTLASS schedule/epilogue pairing

Rule. Mainloop schedule and epilogue schedule must match. A residual-fused kernel with TmaWarpSpecializedCooperative epilogue requires a cooperative mainloop schedule.

Enforcement. The v3 catalog uses a CUDA `static_assert` on the paired schedule types at template instantiation; the variant table generator writes Rust and CUDA sides together, so adding a mismatched variant fails compilation. In v2, the o_proj residual dispatch pins to the hand-verified variant `v1` ($128 \times 128 \times 128$ Coop/Coop), and the catalog has been audited for pairings—the offending variants are cutlass_fp8_gemm_v$\{0, 2, 3, 4, 6, 8, 11, 12, 13, 14\}$ and cutlass_fp8_gemm_residual_v$\{0, 2, 4, 5, 6\}$.

5.5 No `unwrap()` or `String` errors across crates

Rule. All errors crossing crate boundaries are `Result<T, RvllmError>`. The error variants carry structured context—stream identifier, kernel name, launch configuration, GEMM shape, workspace size needed vs given—not a stringified `DriverError`.

Failure mode. During the April 15 debugging pass, several downstream error paths presented the root-cause `cuGraphLaunch failed` as the opaque string `"dtoh event sync: DriverError(CUDA_ERROR_I ...)"`. The failing kernel name, stream identifier, bucket size, and kernel-launch arguments were discarded at the first `map_err`. Recovery required `compute-sanitizer` to re-derive what the error path should have carried.

6 Measured recovery, April 16, 2026

Three root-cause fixes, in three commits, took the engine from 9,531 tok/s to 19,287 tok/s at $N = 128$ in one working day. None were stopgaps; each eliminated a structural failure mode documented in Section 5.

Commit	Fix	**Δ**(N=128)
`6dabb76a5`	Track `last_padded_batch`; only patch metadata when bucket matches	crash → runs
`31716269d`	Re-enable fused FP8 o_proj + residual on variant `v1` (Coop/Coop)	+83%
`eb9e247fd`	Real block-change detection via `block_table_update` (not CoW-only)	+10%

In addition, `libfa3_kernels.so` was built on the deploy host (23 MB, once), eliminating a silent `.ptx` attention fallback path. All four changes taken together restore the engine to a configuration where every decode step uses the intended kernels, with no silent degradation in the dispatch.

7 Results

7.1 Experimental setup

All measurements in this section are on a single NVIDIA H100 SXM 80 GB (driver compatible with CUDA 12.4), running Qwen2.5-7B-Instruct from the public HuggingFace checkpoint, quantized to FP8 E4M3 at engine startup with per-tensor scale (clamp-ppm gate enforced). The v3 benchmark harness is `v3/crates/rvllm-bench`, driven by `RVLLM_*` environment variables; it runs 30 iterations per batch size after 5 warmup iterations and reports tokens per second as (iters × batch)/elapsed. Greedy decoding, CUDA graph captured once per bucket and replayed per iteration, all FP8 linears dispatched via `cublasLtMatmul`, `libfa3_kernels.so` built from the Hopper FlashAttention-3 source.

7.2 v3 throughput by batch size

N	tok/s	ms/step
32	6,644	4.82
64	12,615	5.07
128	**22,496**	**5.67**

7.3 Head-to-head at $N = 128$: v3 vs vLLM vs v2 archive

Engine	tok/s	ratio to v3
vLLM 0.19 V1 (FP8, FlashInfer, CUDA graphs)	19,399	0.862×
rvllm-v2 (archive, `eb9e247fd`)	19,287	0.858×
rvllm-v3 (fair bench)	**22,496**	**1.00×**

The vLLM measurement is taken on the same H100 SXM 80 GB box that runs v3, using the same `Qwen2.5-7B-Instruct` checkpoint and the same FP8 quantization setting, via `vllm bench latency -model Qwen2.5-7B -quantization fp8 -batch-size 128 -input-len 16 -output-len 512 -num-iters 3 -num-iters-warmup 1 -dtype float16`. The reported number is vLLM's engine log line `Avg generation throughput`, a steady-state decode metric that excludes prefill by construction. vLLM's total end-to-end latency for the 128-prompt, 512-output-token batch was 3.243 s.

What's 1:1 in this comparison. Same GPU, same model weights, same FP8 E4M3 quantization type, same batch size, CUDA graphs enabled in both. Both figures are decode tokens per second reported by each engine's own steady-state metric.

The three original caveats, and how v3's bench now addresses each. The initial v3 number at 22,559 tok/s raised three legitimate concerns about the fairness of a direct comparison against vLLM's serving measurement. We modified the v3 bench to address each one and re-ran:

1. **vLLM does real prefill** (16 input tokens × 128 prompts = 2,048 tokens of prompt processing before decode starts). The v3 bench skips a dedicated prefill kernel. vLLM's `Avg generation throughput` metric is decode-only, so this is mostly accounted for. *Fix:* the v3 bench runs 16 eager decode steps before the timed window, which populate KV pages with real forward-pass activations. This matches the KV-cache state vLLM has at the start of its decode measurement.

2. **vLLM runs its scheduler every step.** At steady-state batch=128 it still checks request state, completions, and block-table management. The v3 bench previously replayed a frozen graph. *Fix:* the v3 bench now re-uploads `positions`, `slot_mapping`, and `context_lens` every timed iteration. v3 pays the same per-step HtoD cost that vLLM's scheduler pays for metadata management.

3. **vLLM's KV cache has real content from real prompts.** v3's had post-warmup garbage. *Fix:* positions/slot_mapping/context_lens advance across the 16-step faux-prefill and the 30 timed decode steps, so the paged KV progressively fills with real activations.

After all three fixes, v3 measures **22,496 tok/s** (vs. 22,559 on the frozen-graph version), a 0.3% decrease. The per-step metadata upload and faux-prefill are small relative to the 28-layer forward pass and LM head. v3 remains 16% faster than vLLM, confirming the edge is structural (fused QKV, cuBLASLt autotune, fused epilogues, no Python dispatch) rather than an artifact of benchmark setup.

Sources of v3's gain over v2. Four structural changes: (i) fused Q‖K‖V GEMM (one launch, $N = 4608$) versus v2's three separate Q/K/V GEMMs; (ii) `cublasLtMatmul` replacing hand-picked CUTLASS variants on all five linears (QKV, O, gate_up, down, lm_head); (iii) QKV bias fused into the GEMM epilogue via `CUBLASLT_EPILOGUE_BIAS`; (iv) O and down residual-add fused into the GEMM via `beta`=1 against the residual tensor. Each change removes a kernel launch without introducing a megakernel.

7.4 v3 session progression

Step	tok/s	Δ
Eager (no graph)	551	—
+ CUDA graph capture	14,745	27×
+ f16 RoPE tables	15,537	+5%
+ q/k/v bias load + fused QKV	15,985	+3%
+ cuBLASLt + `EPILOGUE_BIAS` (QKV)	17,562	+10%
+ all 5 linears on cuBLASLt	**22,496**	**+28%**

7.5 Where the cycles go (v2 baseline reference)

Per-kernel time measured on v2 at $N = 32$ (nsys profile); reproduced here for reference since v3's cuBLASLt path invokes different algorithms not directly characterized by v2's profiles:

- CUTLASS FP8 GEMM (fused residual, o_proj): 15.9–65.6 μs depending on bucket and variant.
- FA3 SM90 paged decode: 7.5 μs per layer at $N = 32$.
- fused RMSNorm + FP8 quant (per-token): 6.0–7.3 μs.
- fused SiLU*mul + FP8 quant: 14.3 μs.
- cuBLAS LM-head HGEMM: $\sim$483 μs on $(32, 152064, 3584)$.

In v3 the LM-head is FP8 GEMM via cuBLASLt with the activation pre-quantized, removing the HGEMM cost.

8 Related work

vLLM [1] introduced the paged-KV cache and continuous batching patterns that rvLLM's scheduler implements. The paged KV layout, block-manager structure, and preemption semantics follow vLLM's conventions closely; the departure is the Rust runtime, the graph-replay discipline, and the explicit rejection of the dispatch fallback chain.

FlashAttention-3 [**?**] supplies the SM90 paged-decode kernel; rvLLM links directly against a `.so` built from the Hopper source tree and does not re-implement the attention kernel. The `.so` build, variant selection, and workspace contract are rvLLM's responsibility.

CUTLASS [**?**] supplies the FP8 GEMM templates. rvLLM generates the variant catalog, enforces the schedule-pairing rule, and ships a per-shape autotune policy as a build artifact; the CUTLASS library itself does not enforce pairings or manage workspaces across a runtime.

TensorRT-LLM [**?**] and SGLang [**?**] occupy the same space but both assume a CUDA-Python host process. rvLLM's aim is to demonstrate that a GPU-serving engine can be written entirely in a systems language with no Python interpreter in the critical path, and that doing so exposes the kernel-level correctness invariants that a CUDA-Python host can paper over.

9 Limitations and future work

The engine at `eb9e247fd` is single-GPU only. Multi-GPU tensor parallelism exists in a sibling crate but is not wired into the v2 FP8 path. Pipeline parallelism is not implemented. The model catalog validated end-to-end on this FP8 path is Qwen2 family, Llama 3 family, Mistral 7B, and Gemma 2; other supported architectures compile but have not been validated against HuggingFace reference on this version. Speculative decode, LoRA serving, and tool-calling guided decoding are present in the Python-free API layer but outside this paper.

The v3 rewrite consolidates the engine into 16 crates with a DAG-enforced dependency graph and per-crate LoC budgets, and makes each of the five invariants in Section 5 either a compile error, a type-level rejection, or a single-source-of-truth runtime check. The CUDA `static_assert` on schedule pairings, the removal of the second metadata upload path, the `GraphSafe` marker trait rejecting `&mut HbmArena` inside `CaptureScope` closures, and the SHA-pinned `policy.json` artifact each eliminate one of the three bug classes documented above. v3 is tracked under `v3/SPEC.md` and `v3/IMPL_PLAN.md` in the repository.

10 Conclusion

rvLLM demonstrates that an LLM inference engine competitive with modern Python-hosted serving stacks can be written entirely in Rust, with a single compute stream per worker, a frozen metadata layout per graph bucket, and a kernel catalog in which every variant's workspace and schedule are known at engine initialization. The v3 engine achieves **22,496 tok/s** serving Qwen2.5-7B in FP8 on a single H100, with 339 kernel launches per decode step compressed into one `cuGraphLaunch` replay. The archived v2 stack (commit `eb9e247fd`) reached 19,287 tok/s at the same batch on the same GPU.

The path to the v3 number was mechanical, not magical: capturing the layer loop into a CUDA graph alone moved the engine from 551 tok/s (eager) to 14,745 tok/s (replayed). Fusing Q/K/V into a single GEMM, loading attention biases that v2 had silently omitted, and routing every FP8 linear through `cublasLtMatmul` with bias and residual epilogues took the engine the rest of the way.

The three SM90 / graph-capture failure modes documented in Section 5 — producing either silent correctness bugs or `CUDA_ERROR_ILLEGAL_ADDRESS` only under graph replay — are made unrepresentable in v3's type system or caught at compile time. A fourth class of silent bug surfaced during v3 development: omitting model components assumed by the architecture (here, Qwen2.5's attention biases). The invariant added for v3 is that every tensor listed in the model's safetensors index must be consumed by the loader or the engine refuses to start.

References

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.